Experiment 9: Eight Puzzle using Hill Climbing

Aim:

Program to implement a Game Playing algorithm for eight puzzle program/ Graph.

Theory:

The **8-puzzle** is a 3 × 3 board with eight numbered tiles and one blank square. A tile next to the blank can slide into it. Equivalently, the blank moves Up, Down, Left or Right, so each state has two to four successors depending on where the blank is. Given a start state and a goal state, the task is to find a sequence of moves that transforms one into the other. From any state, 9!/2 = 181,440 states are reachable, so blind search is wasteful and the problem is a standard example of **heuristic search**.

Heuristic function. A heuristic function h(n) estimates how far a state n is from the goal, and h(goal) = 0. Two common heuristics for the 8-puzzle are:

- **h_1(n)**: the number of tiles out of place (Hamming distance). The blank is not counted.
- **h_2(n)**: the sum, over all tiles, of the Manhattan distance of each tile from its goal position.

The program uses h_1, and for every state it also prints which tiles are misplaced.

Hill Climbing. Hill climbing is a local search algorithm that uses only the heuristic value to decide its next move. From the current node it generates all successors with *MoveGen*, sorts them on h, and moves to the best successor, but only if that successor is better than the current node (for the 8-puzzle, a lower h). It keeps no OPEN or CLOSED list and never backtracks, so it needs only **constant space**, and its time is proportional to the number of moves made. Because it always moves in the direction of greatest improvement, this form is also called **steepest gradient ascent** (or descent, when h is being minimised). The search has turned from finding a goal node into optimising the value of h.

The algorithm stops when no successor is better than the current node. If that node is the goal, the search has succeeded. Otherwise it is stuck in one of these situations:

- **Local optimum:** every neighbour is worse, although the goal lies elsewhere.
- **Plateau:** neighbours have the same heuristic value, so there is no gradient to follow.
- **Ridge:** the better states lie in a direction that no single move can reach.

Hill climbing is therefore **not complete**: even for a solvable puzzle, h_1 may lead it to a local minimum. Its strengths are speed and very low memory use. Variations such as random restarts, stochastic hill climbing, simulated annealing and tabu search exist to escape local optima. For each step the program prints every neighbour with its misplaced tiles and h(n), the chosen move, and finally whether the goal was reached or the search stopped at a local minimum or plateau.

Algorithm:

Deepak Khemani, A First Course in Artificial Intelligence, Figure 3.7.

```
HillClimbing()
1  node ← start
2  newNode ← Head(Sort_h(MoveGen(node)))
3  while h(newNode) < h(node)
4     do node ← newNode
5        newNode ← Head(Sort_h(MoveGen(node)))
6  return newNode
```

Figure 3.7 Algorithm Hill Climbing

MoveGen(node) returns the successors of node (blank moved Up/Down/Left/Right), $Sort_h$ orders them by increasing h, and Head picks the first, i.e. the best, successor.